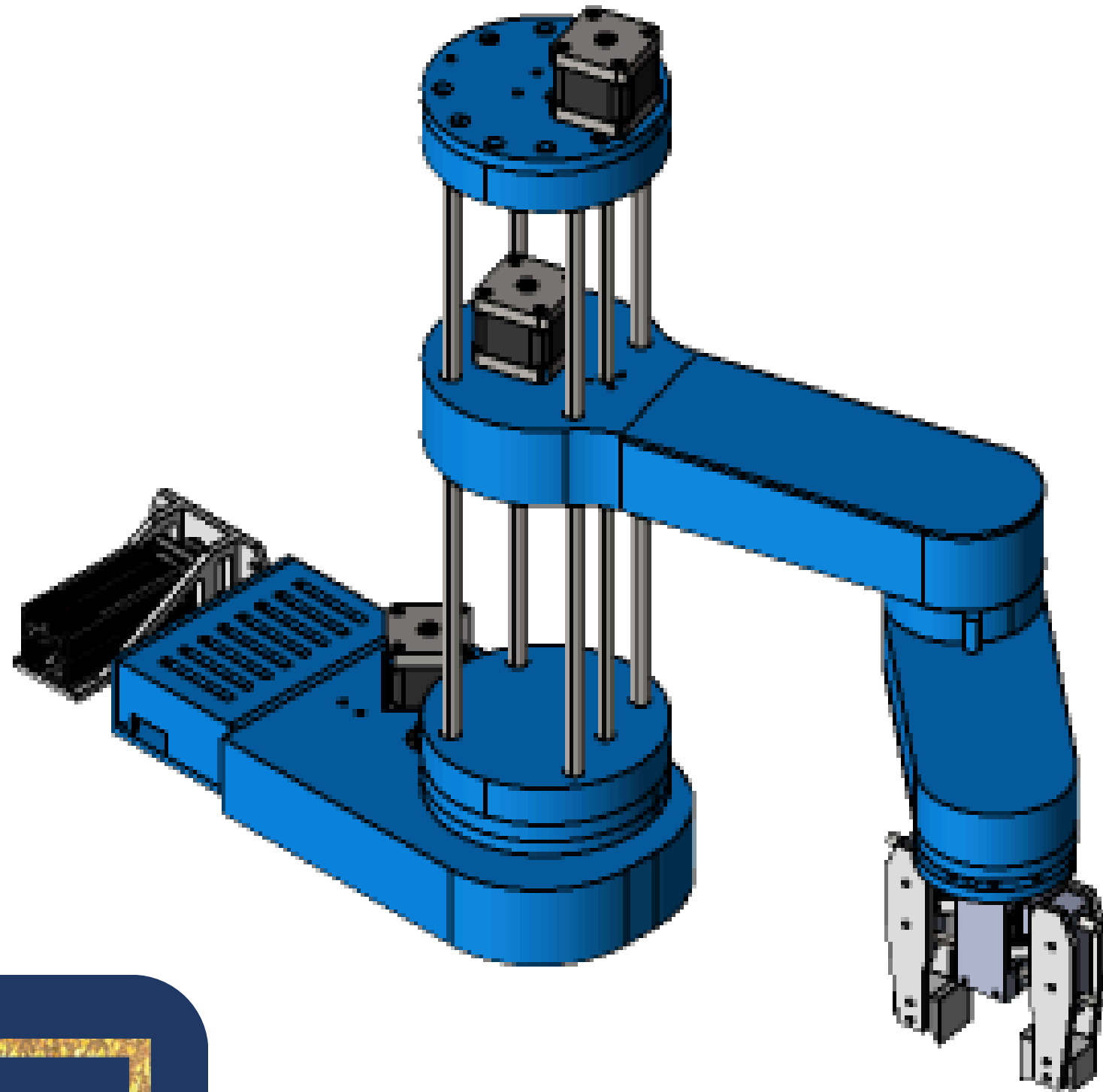
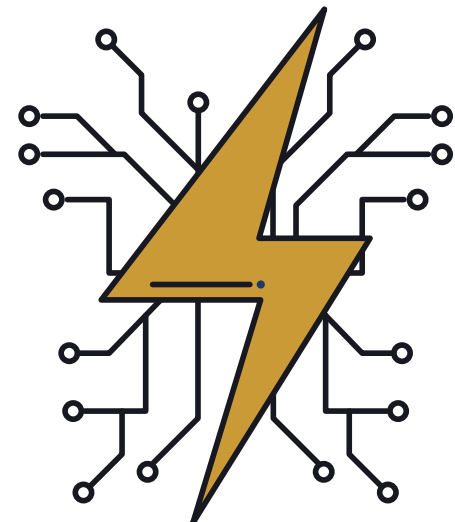


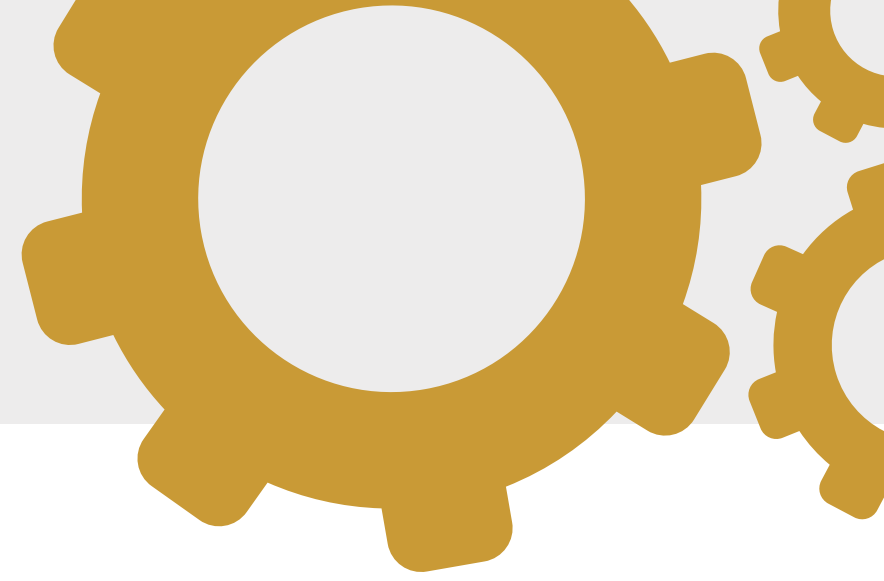


# ROBOT SCARA

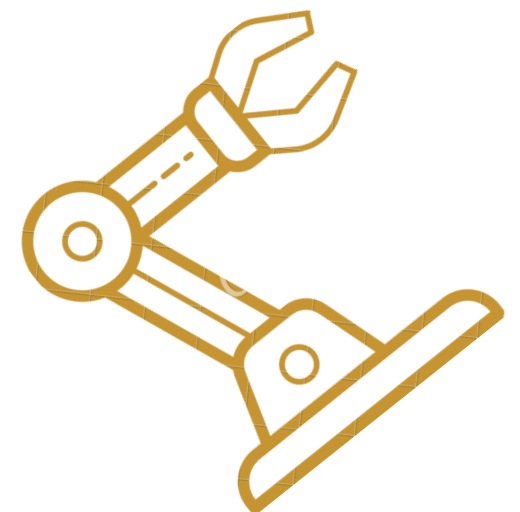
Groupe : 8



# SOMMAIRE



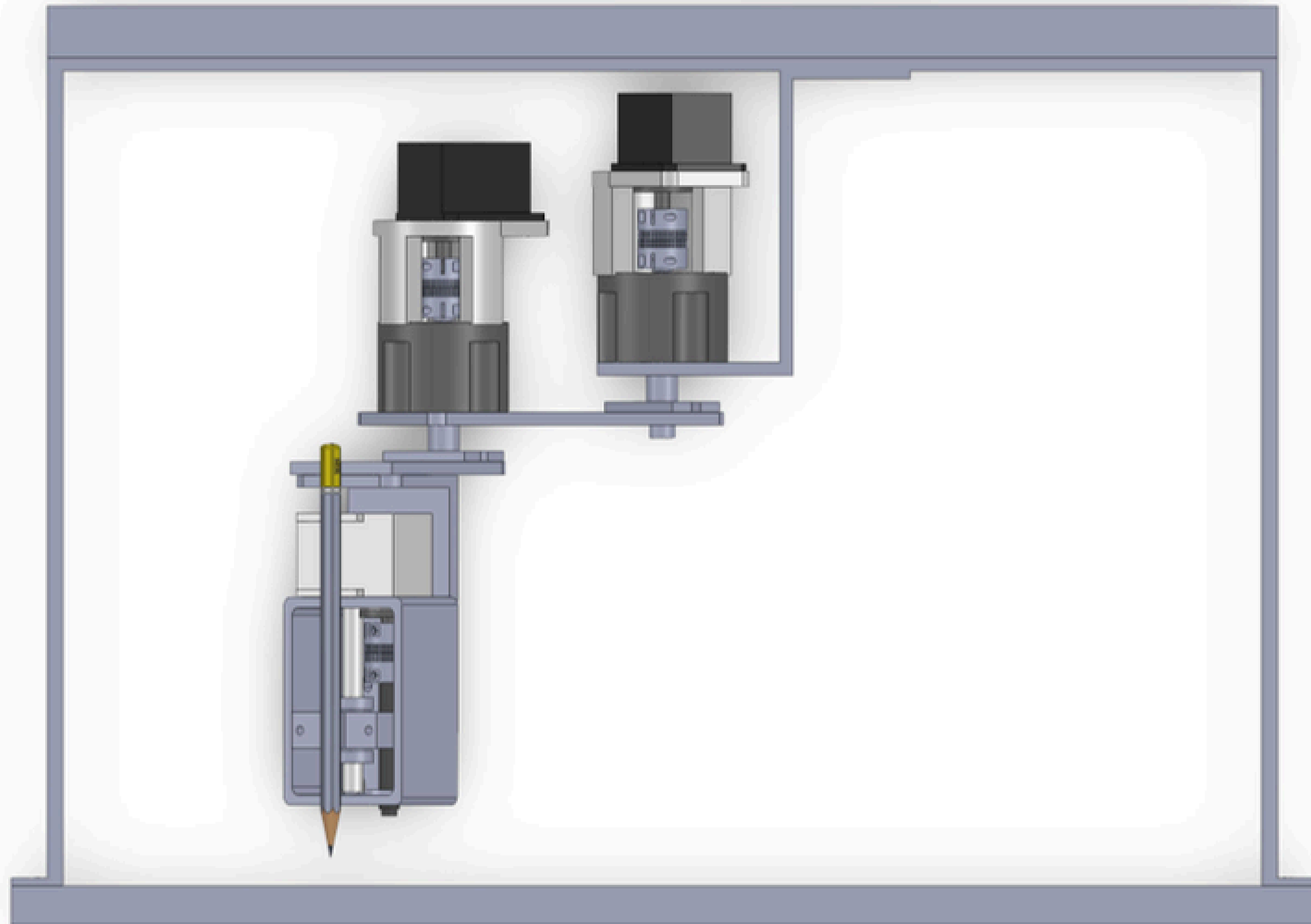
1. *La maquette numérique sur SOLIDWORKS.*
2. *Le schéma électrique du robot.*
3. *Rappel des formules cinématiques du bras du robot.*
4. *Comparaison les performances simulées à celles mesurées en terme d'asservissement en position du bras du robot.*
5. *Code Arduino responsable du pilotage du robot.*
6. *Pilotage du ROBOT SCARA avec une interface graphique*
7. *Commande par AI*

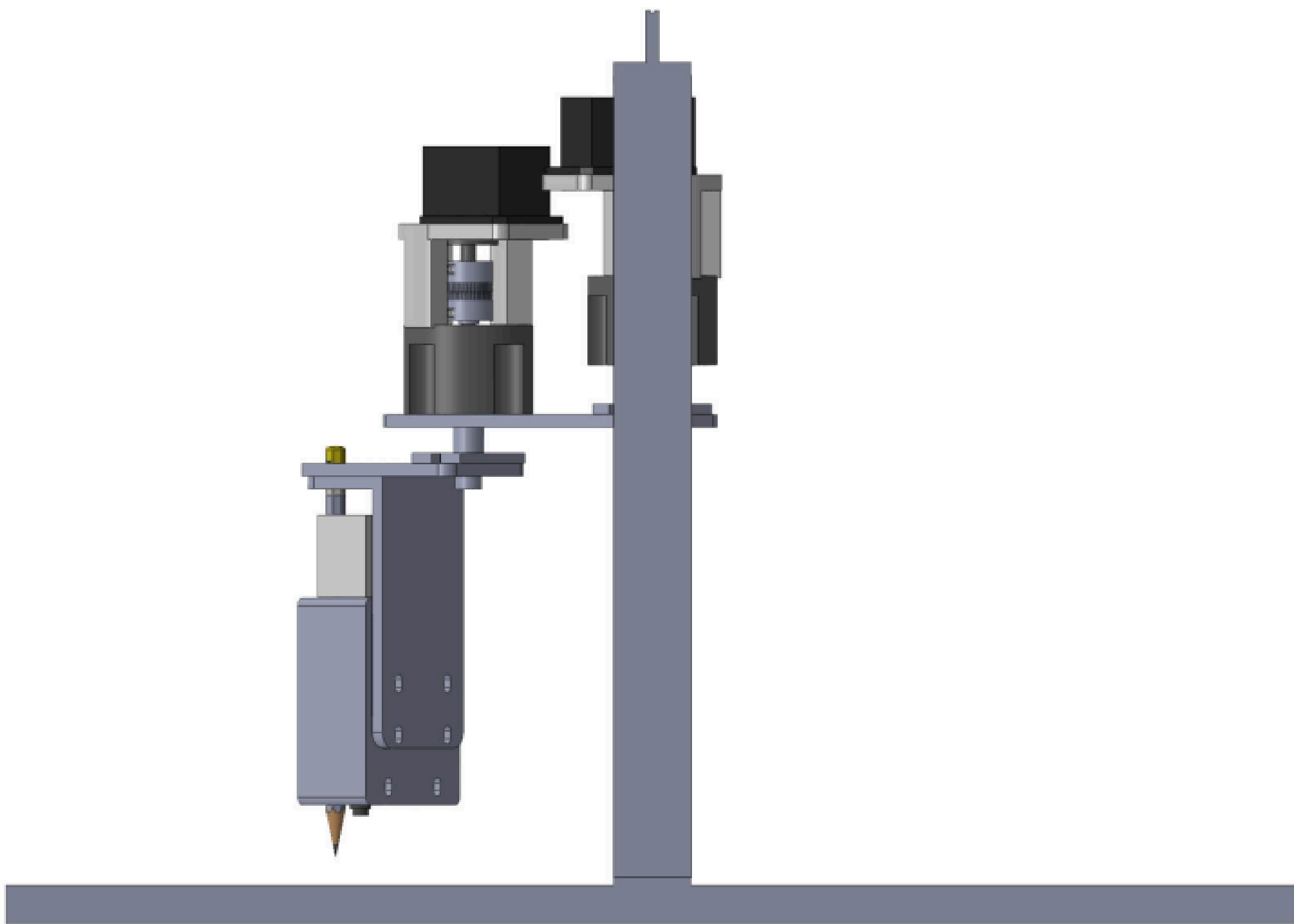


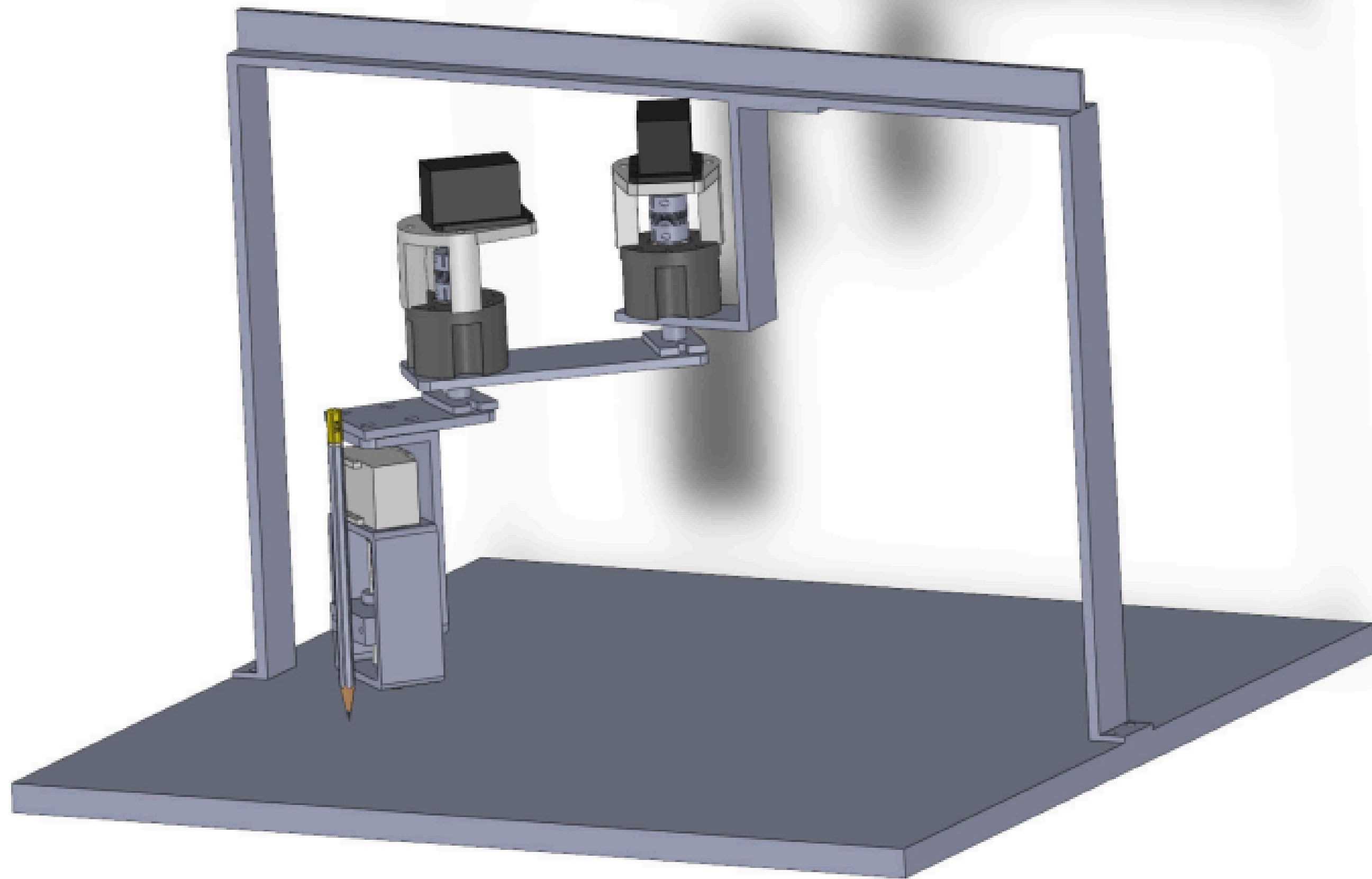


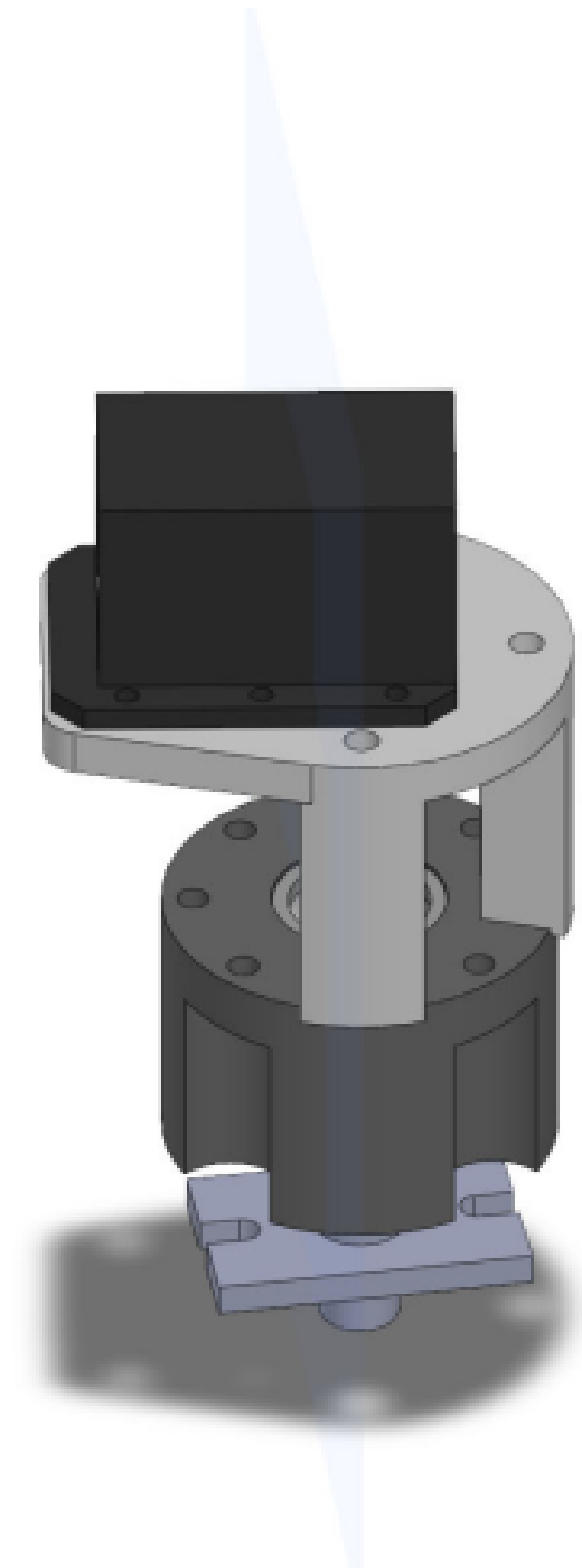
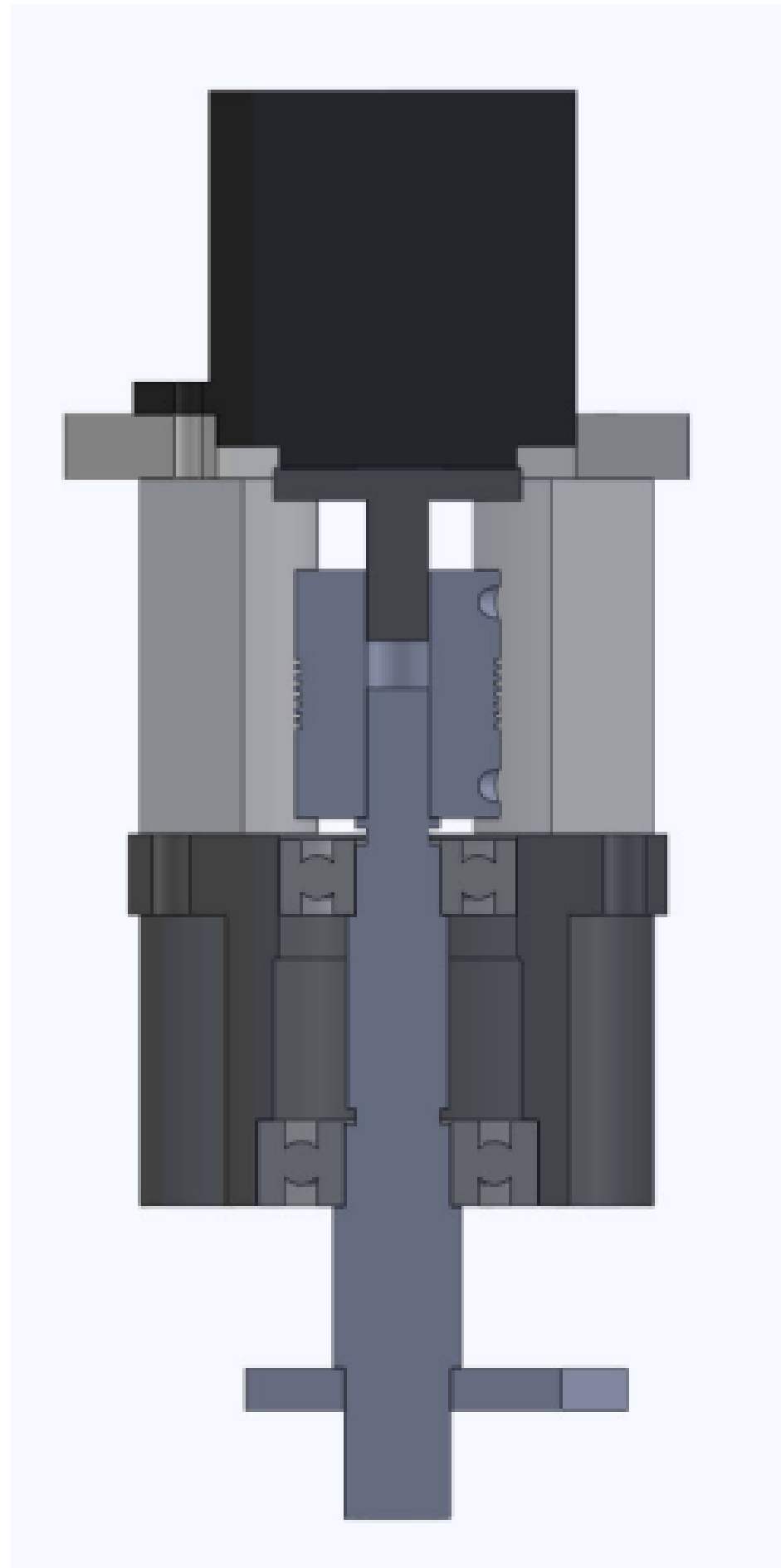
1.

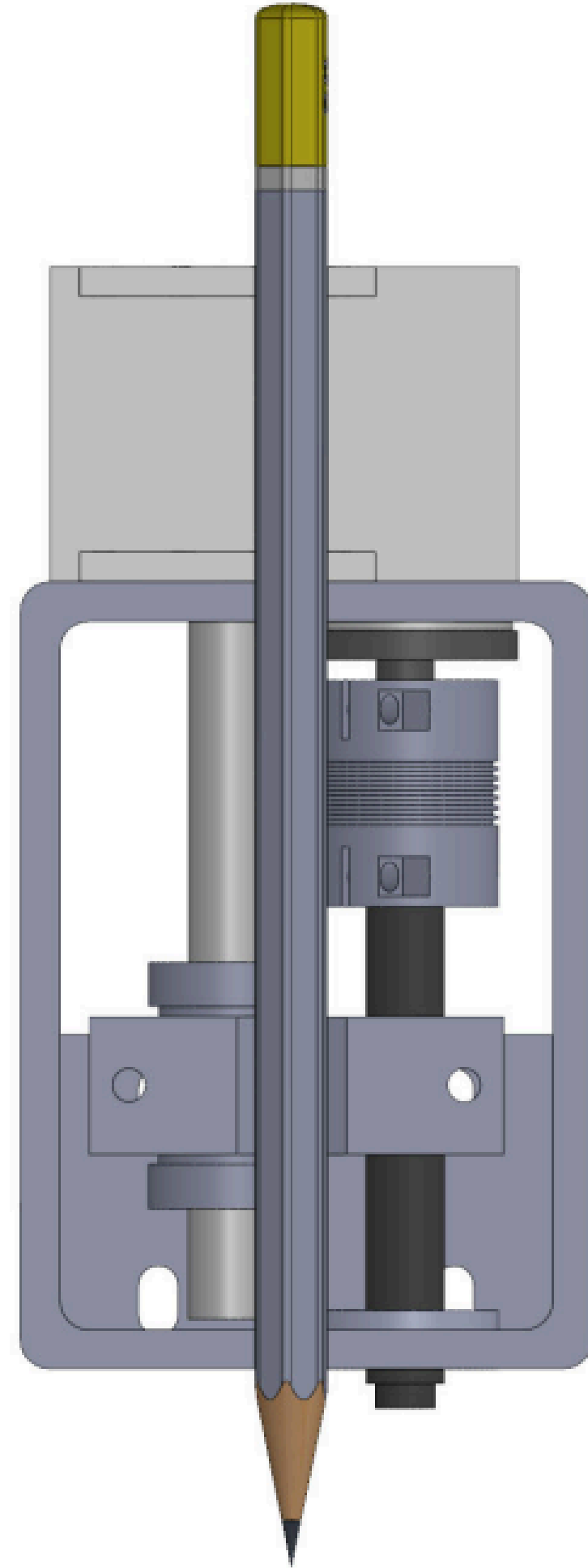
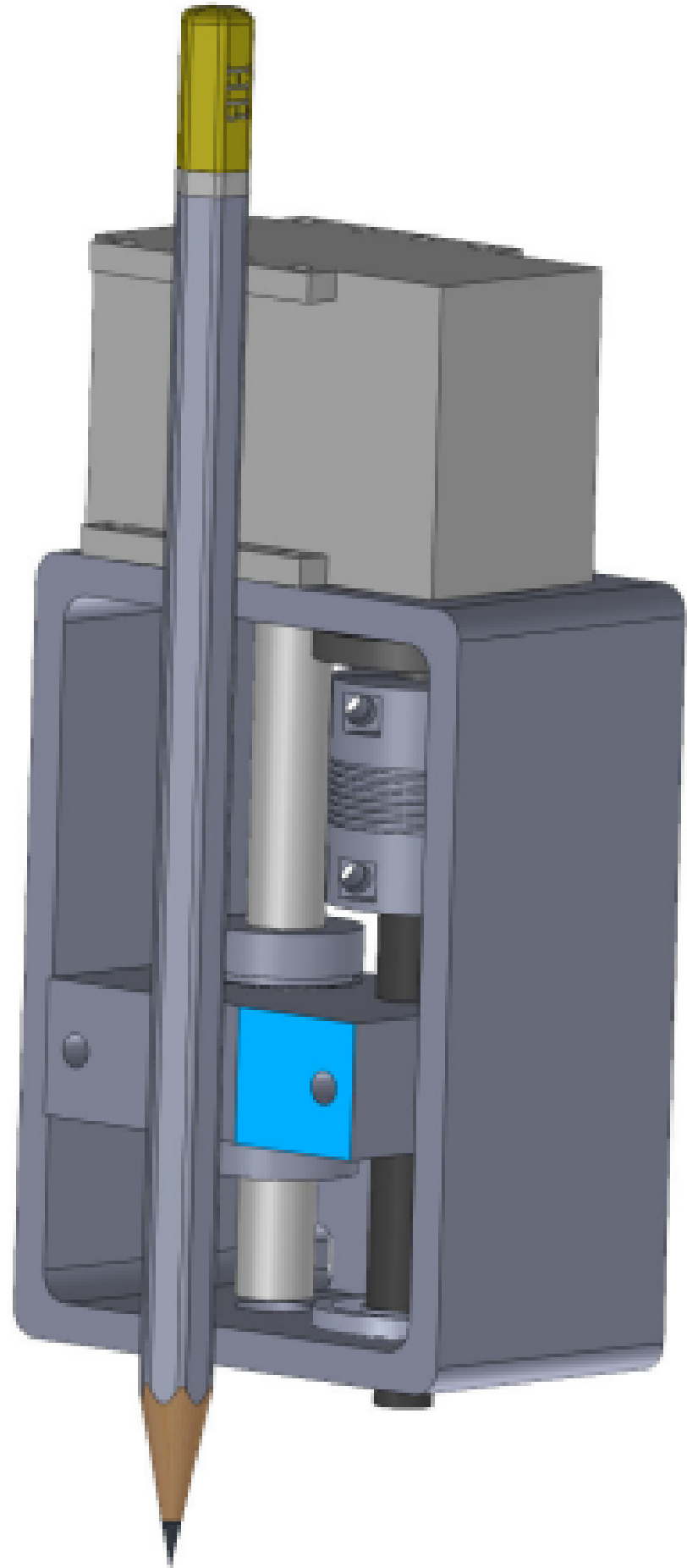
La maquette numérique sur  
**SOLIDWORKS.**

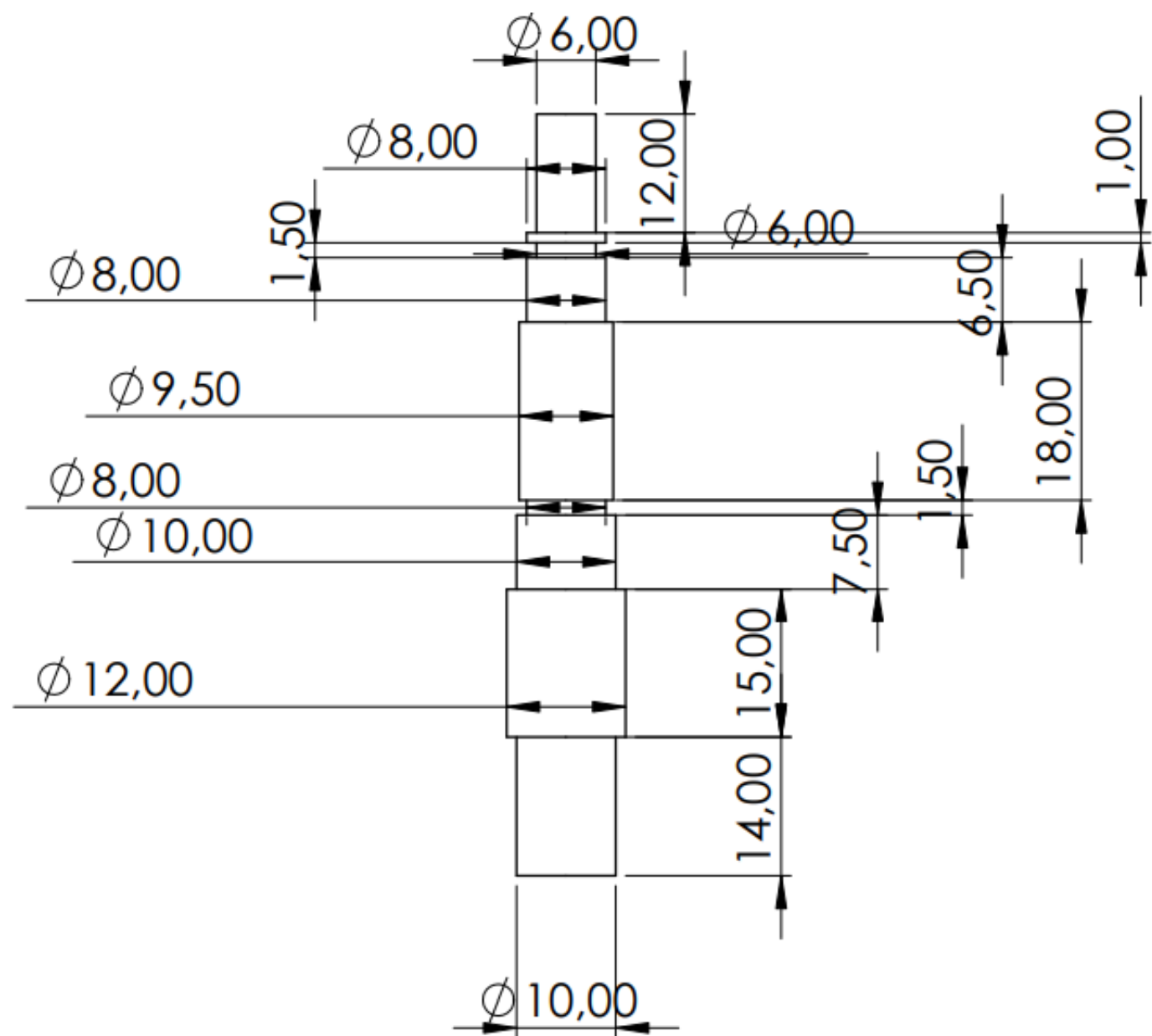




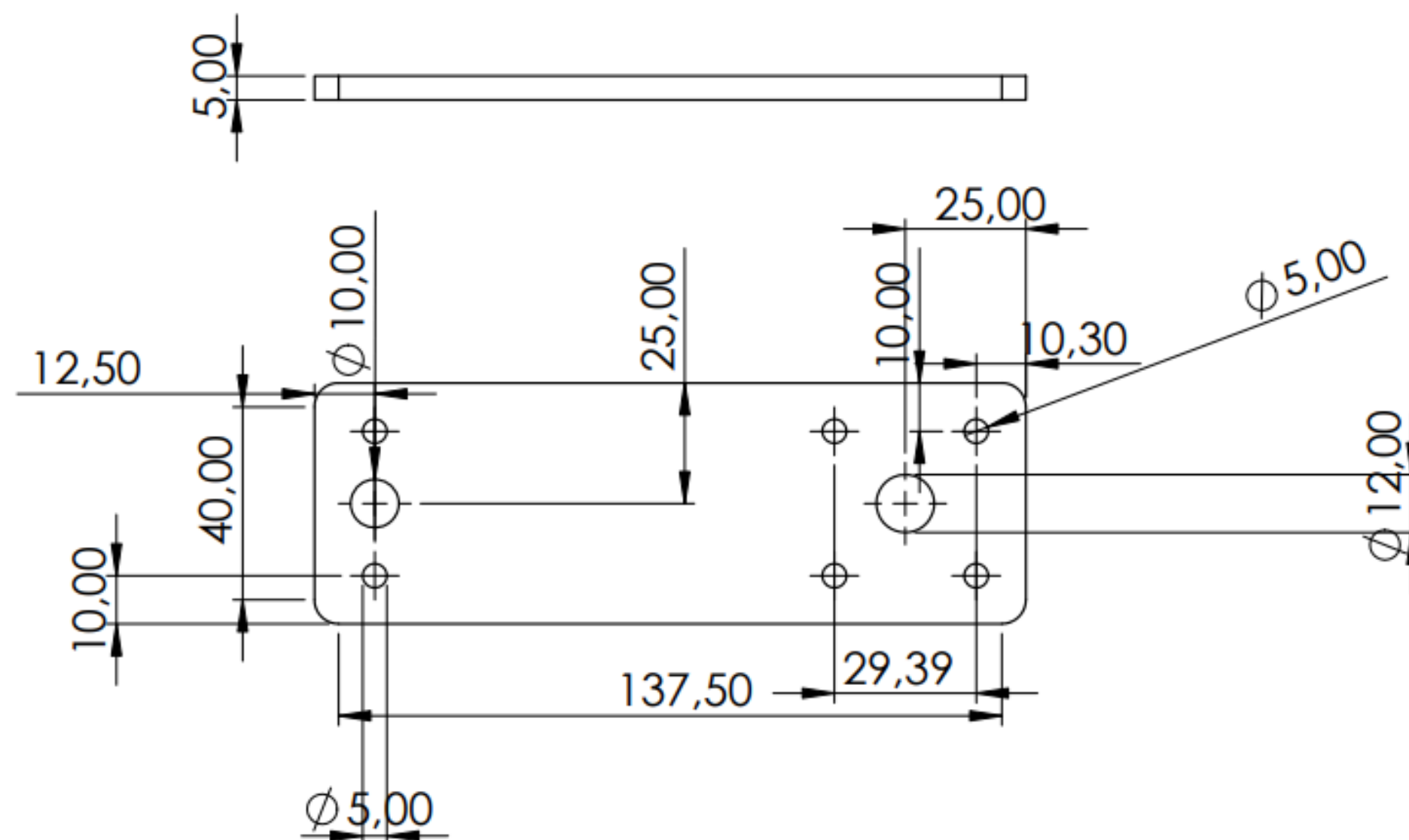








**Axe**

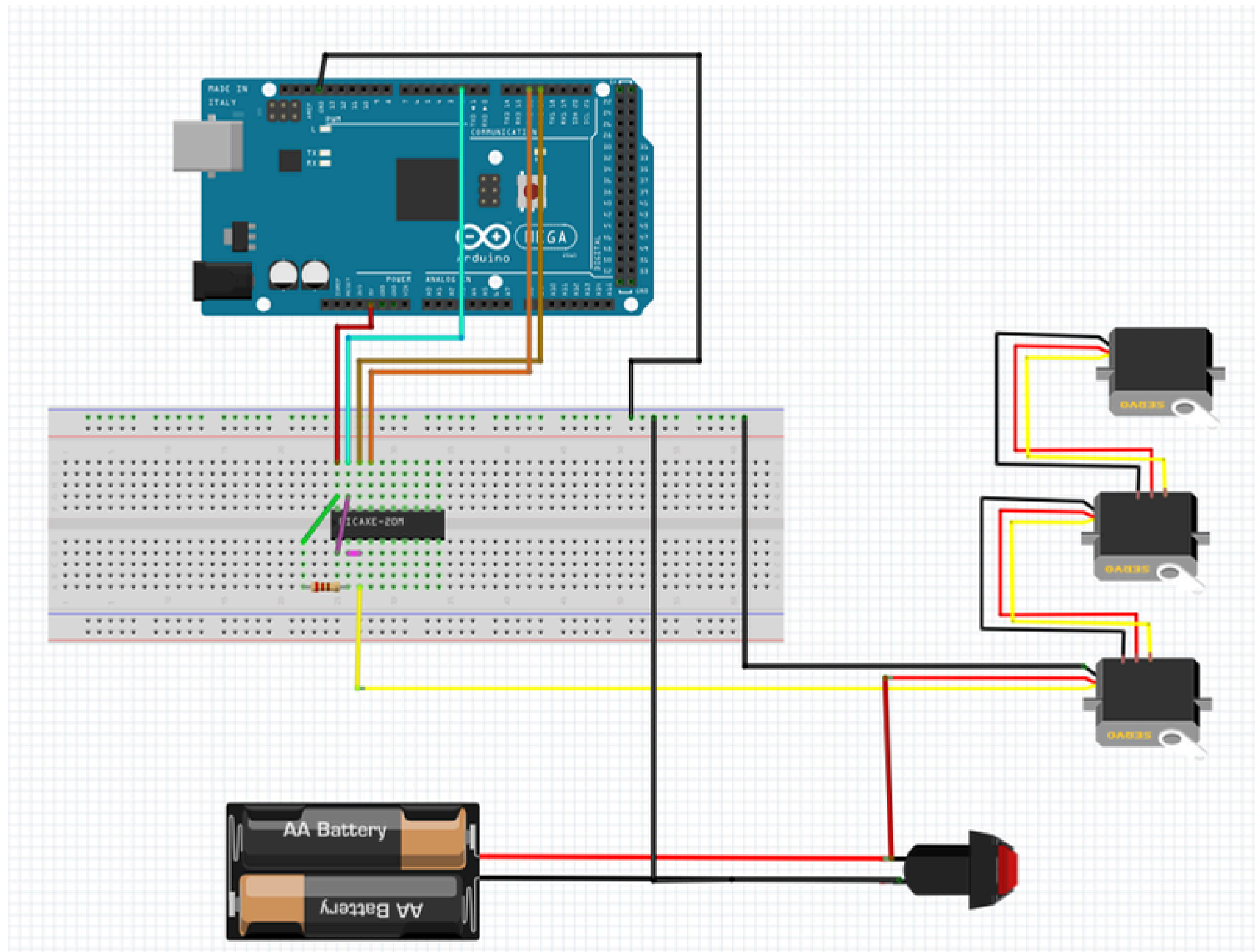


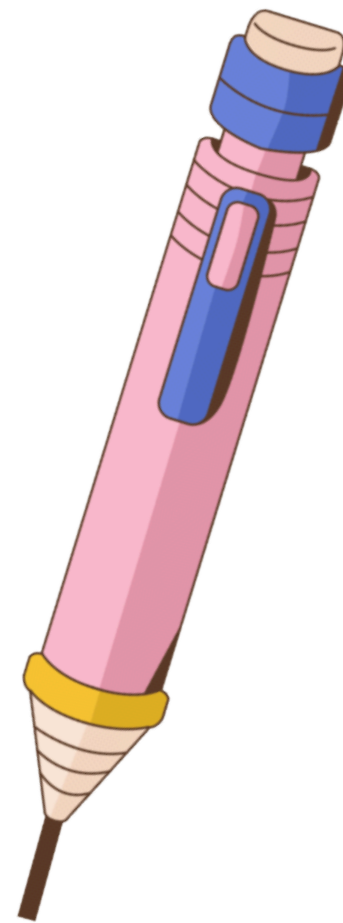
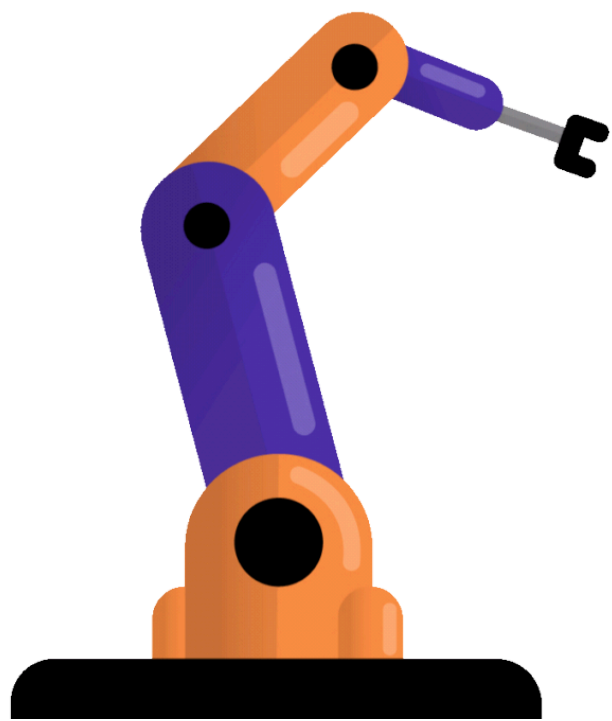
**Bras1**



**2.**

**Le schéma électrique du robot.**





3.

**Rappel des formules cinématiques du bras du robot.**

$$\sum F = 0$$

# Cinématique directe

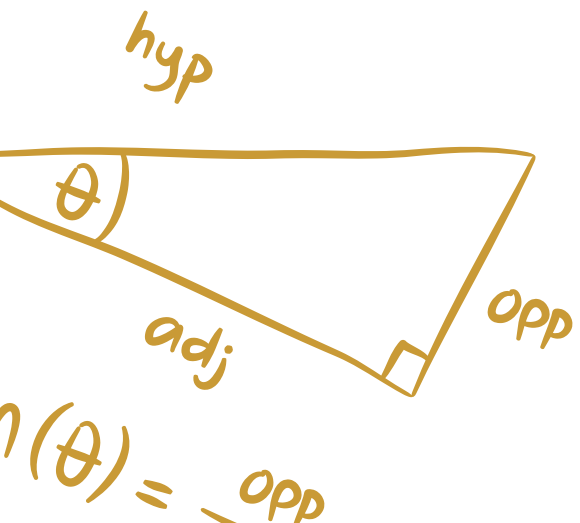
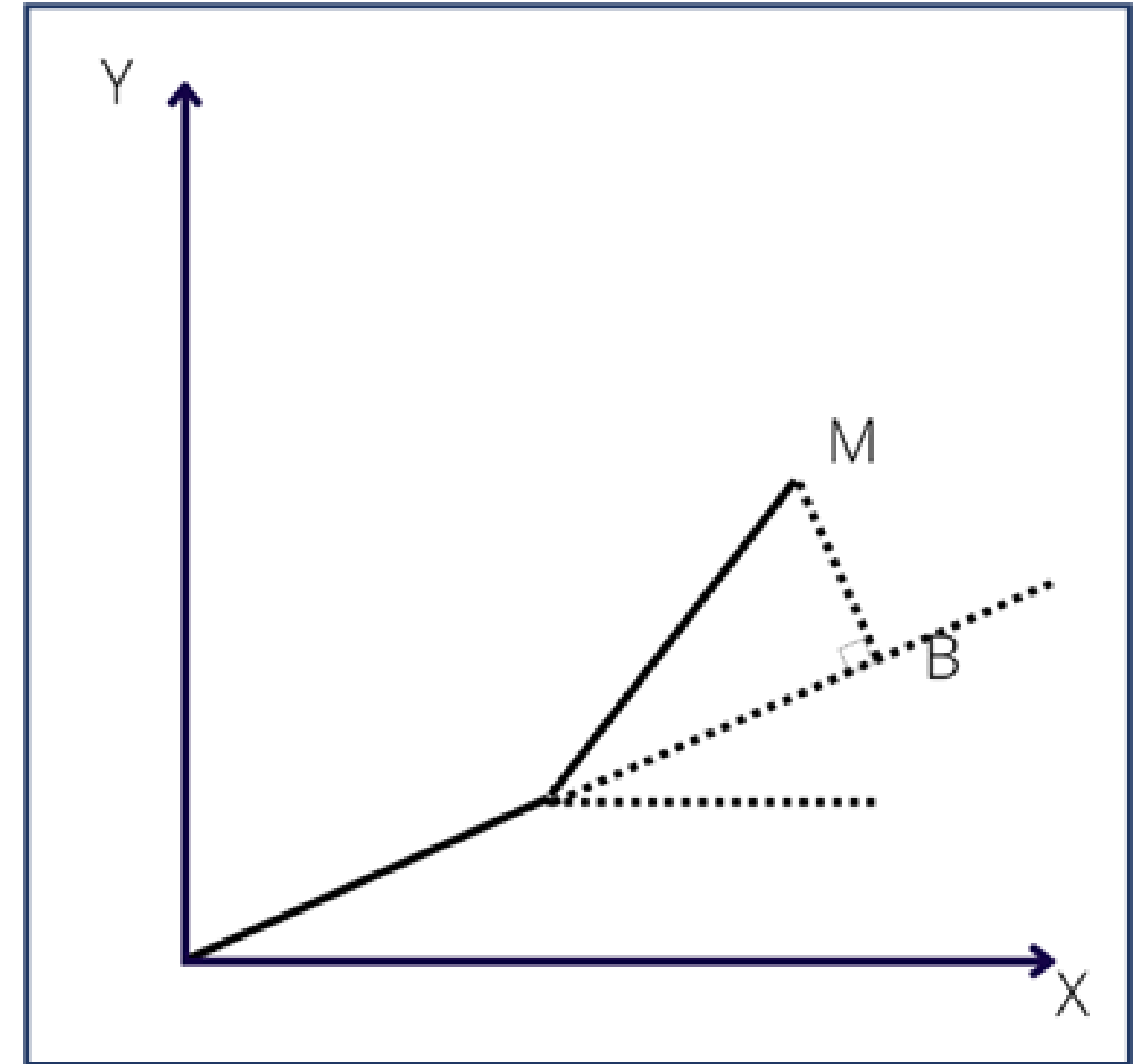
$$x_m = L_1 \times \cos \theta_1 + L_2 \times \cos(\theta_1 + \theta_2)$$

et

$$y_m = L_1 \times \sin \theta_1 + L_2 \times \sin(\theta_1 + \theta_2)$$

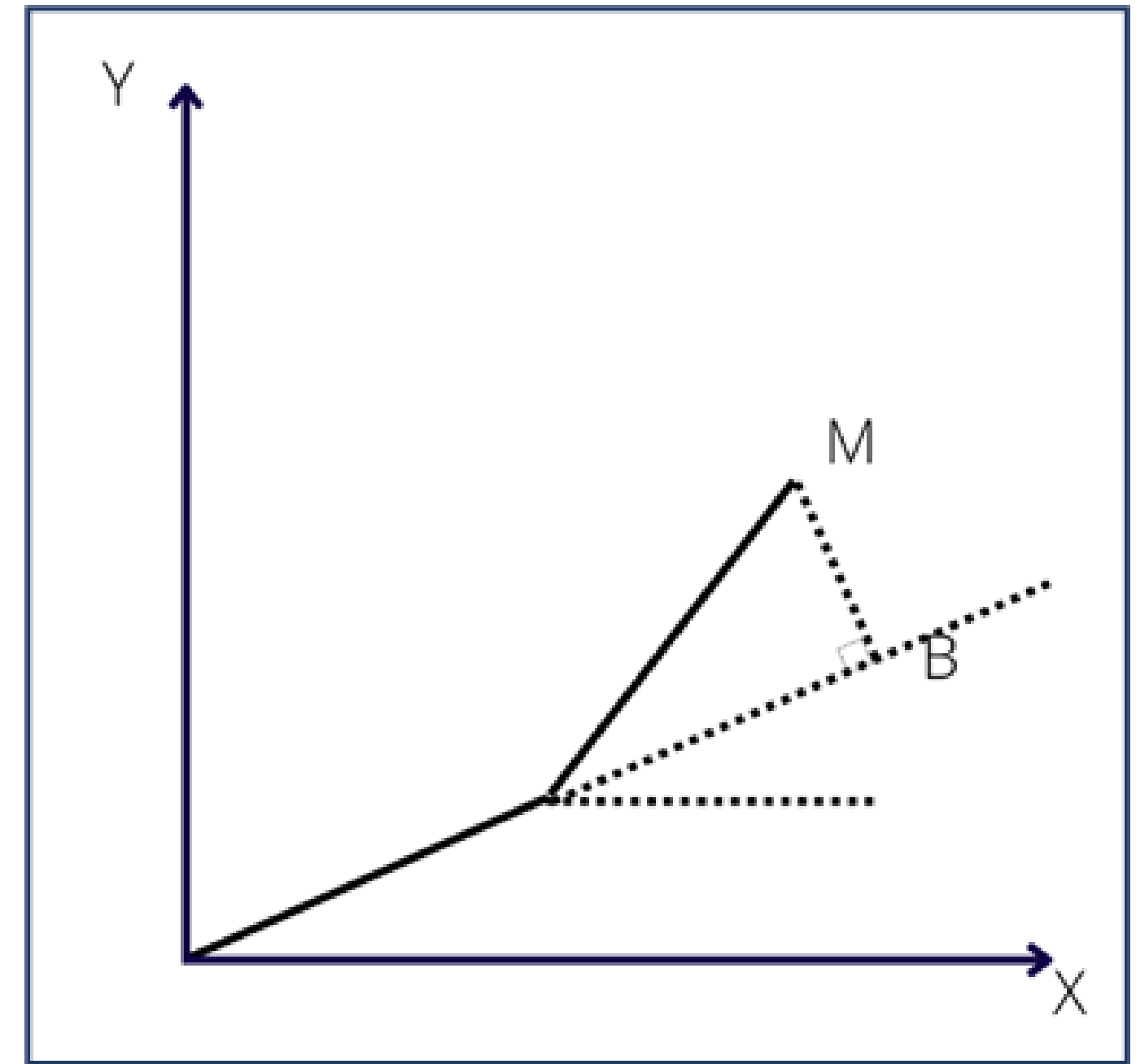
$$x_m = (L_1 + L_2 \times \cos \theta_2) \times \cos \theta_1 - L_2 \times \sin \theta_1 \times \sin \theta_2$$

$$y_m = (L_1 + L_2 \times \cos \theta_2) \times \sin \theta_1 + L_2 \times \cos \theta_1 \times \sin \theta_2$$

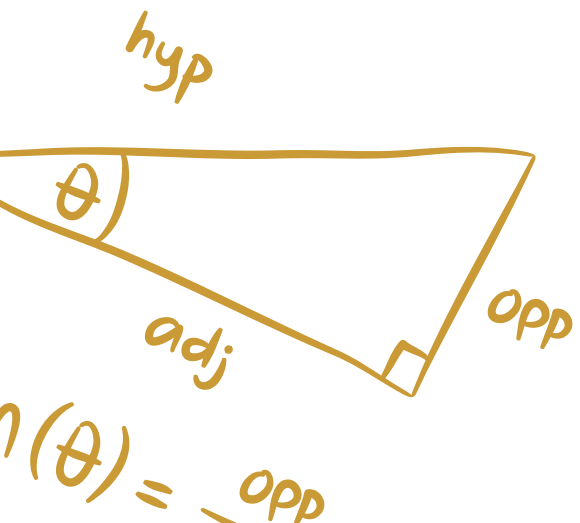


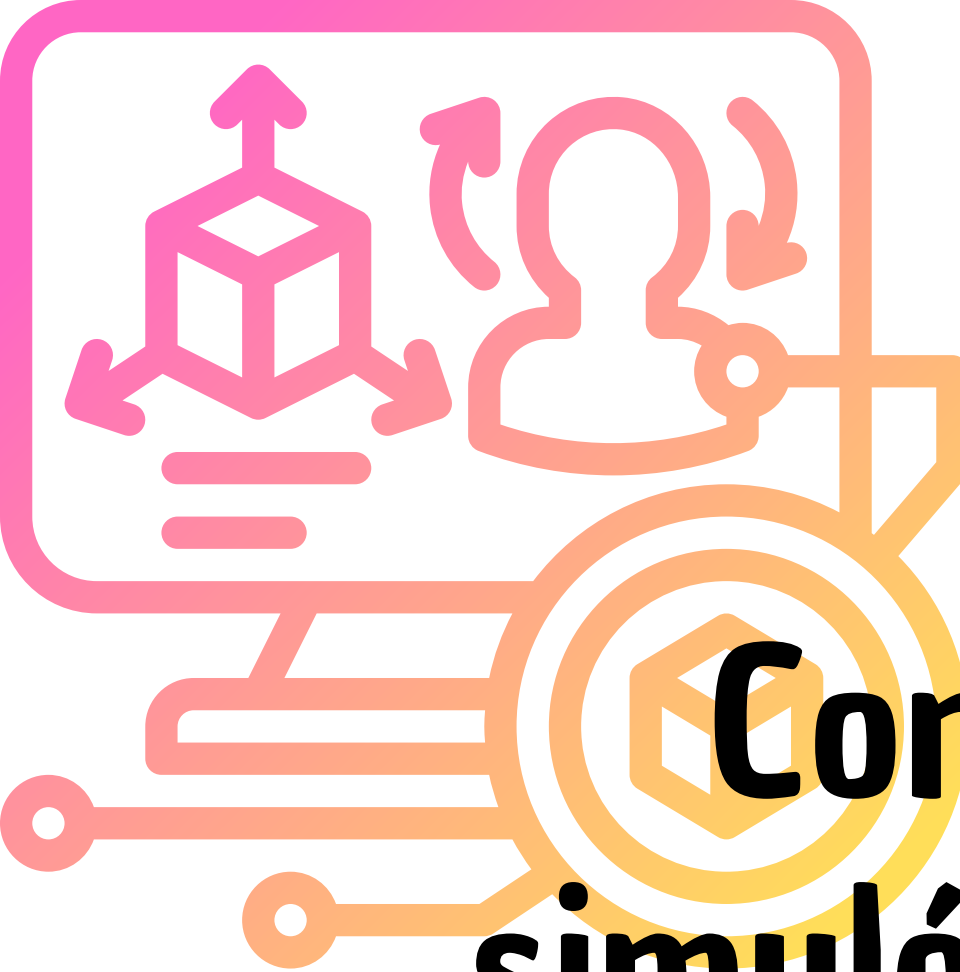
# Cinématique indirecte

$$\cos \theta_2 = \frac{X^2 + Y^2 - L_1^2 - L_2^2}{2 \times L_1 \times L_2}$$



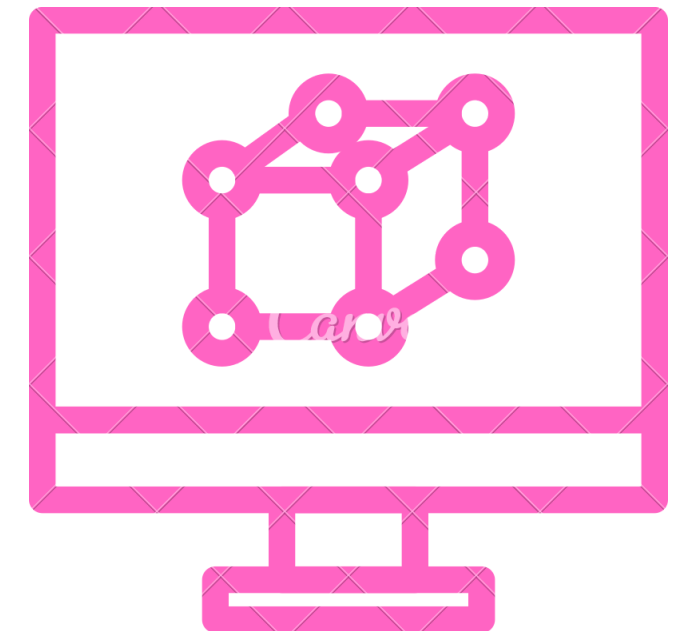
$$\tan \theta_1 = \frac{y_m \times (L_1 + L_2 \times \cos \theta_2) - x_m \times L_2 \times \sin \theta_2}{x_m \times (L_1 + L_2 \times \cos \theta_2) + y_m \times L_2 \times \sin \theta_2}$$

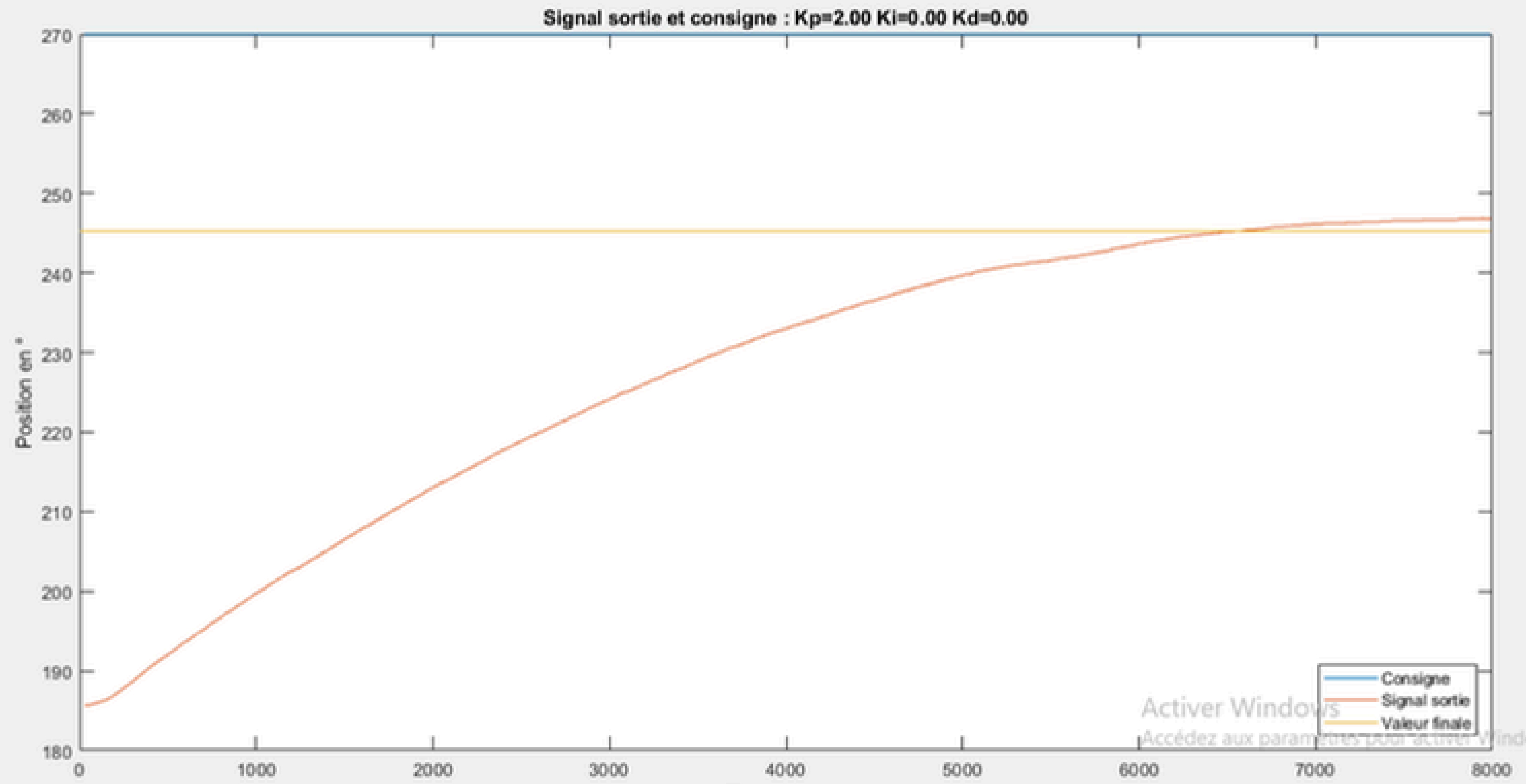




4.

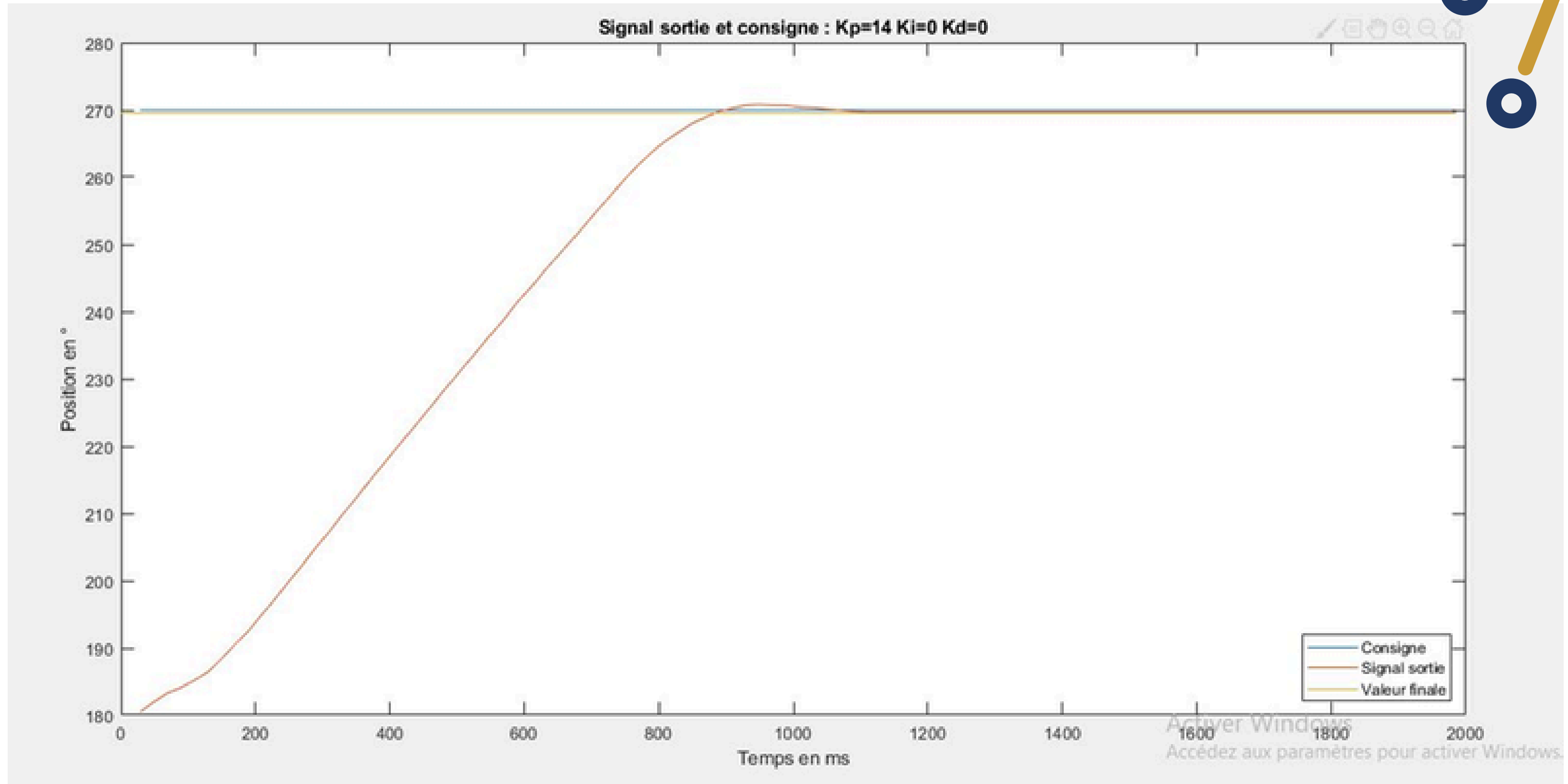
**Comparaison les performances  
simulées à celles mesurées en terme  
d'asservissement en position du bras  
du robot.**

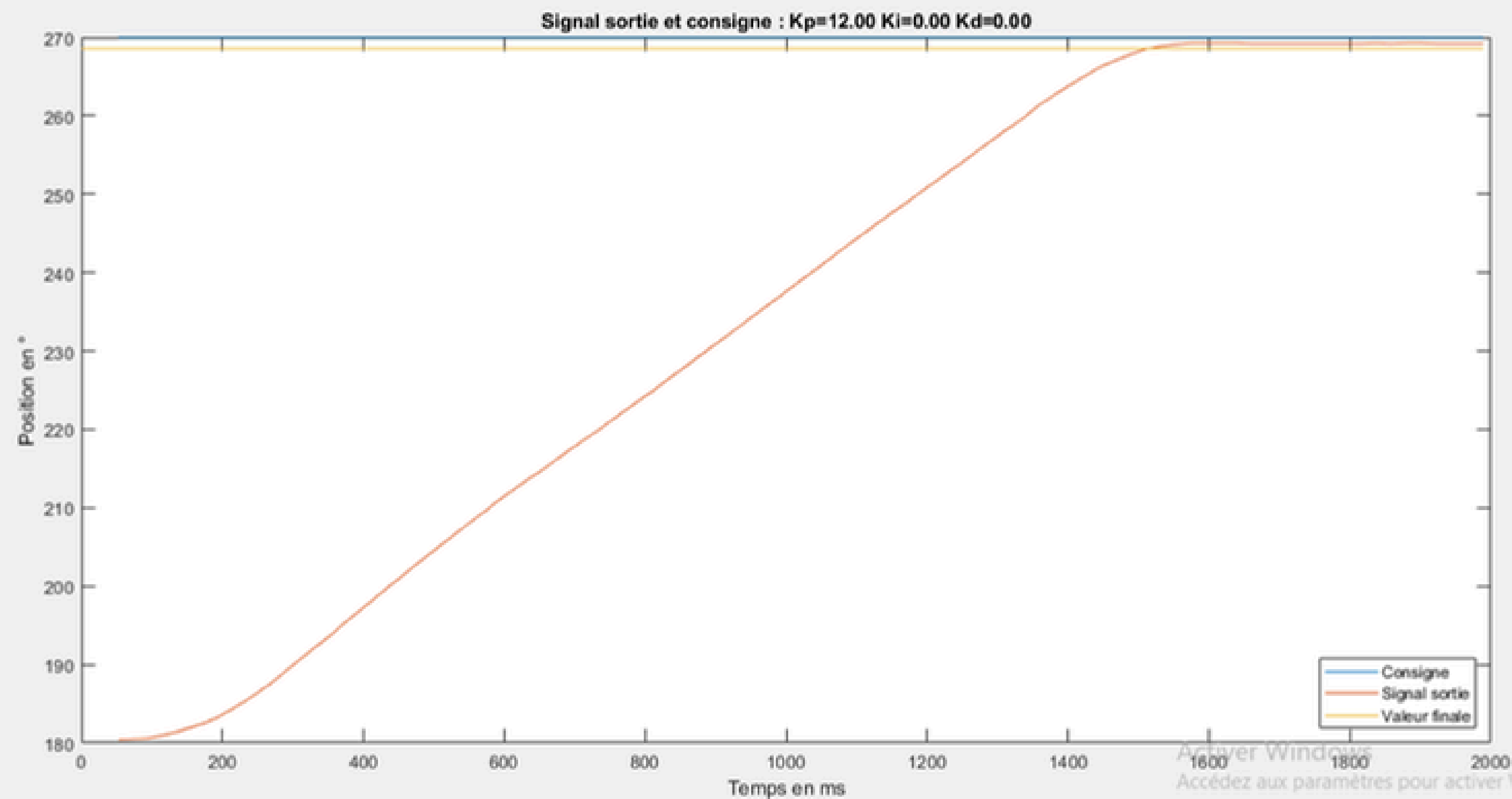




- $K_p$  : P Gain / 8
- $K_i$  : I Gain \* 1,000 / 2,048
- $K_d$  : D Gain \* 4 / 1,000

On choisit donc  $K_p=14$







**5.**

**Code Arduino responsable du pilotage  
du robot.**

```

#include <MX_AX_Serial.h>

int vr=0;

float l1=110.0,l2=110.0;
float  O1 ,O2,O11 ,O22;
float r1,r2;
void setup() {
  Serial.begin(9600);
  Dynamixel.begin(1000000,2);
  delay(1000);
  Dynamixel.setEndless(1,OFF,ON);
  Dynamixel.setEndless(2,OFF,ON);

void loop() {
  while (Serial.available()>0){
    //char rec=Serial.read();
    float x = Serial.parseFloat();
    float y = Serial.parseFloat();
    float z = Serial.parseFloat();
    if (Serial.read() == ('\n')){

      if (z==0 and vr==1){down();}
      if (z==1 and vr==0){up();}

      O2 =  acos((x * x + y * y - (l1 * l1 + l2 * l2)) / (2*l1*l2));
      O1 = atan2((y * (l1 + l2 * cos(O2)) - (x * l2 * sin(O2))) , (x *(l1 + l2 * cos(O2)) + (y * l2 * sin(O2))));
      O22=180.0-(O2*180.0)/(PI);
      O11=270.0-(O1*180.0)/(PI);
      r2=(O22*4095.0)/360.0;
      r1=(O11*4095.0)/360.0;
      Dynamixel.moveSpeed(1,r1,50);
      Dynamixel.moveSpeed(2,r2,50);

    }

  }

  void up() {
    Dynamixel.setEndless(3,ON,OFF);
    Dynamixel.turn(3,LEFT,1000);
    delay(150);
    Dynamixel.turn(3,LEFT,0);
    vr=1;
  }

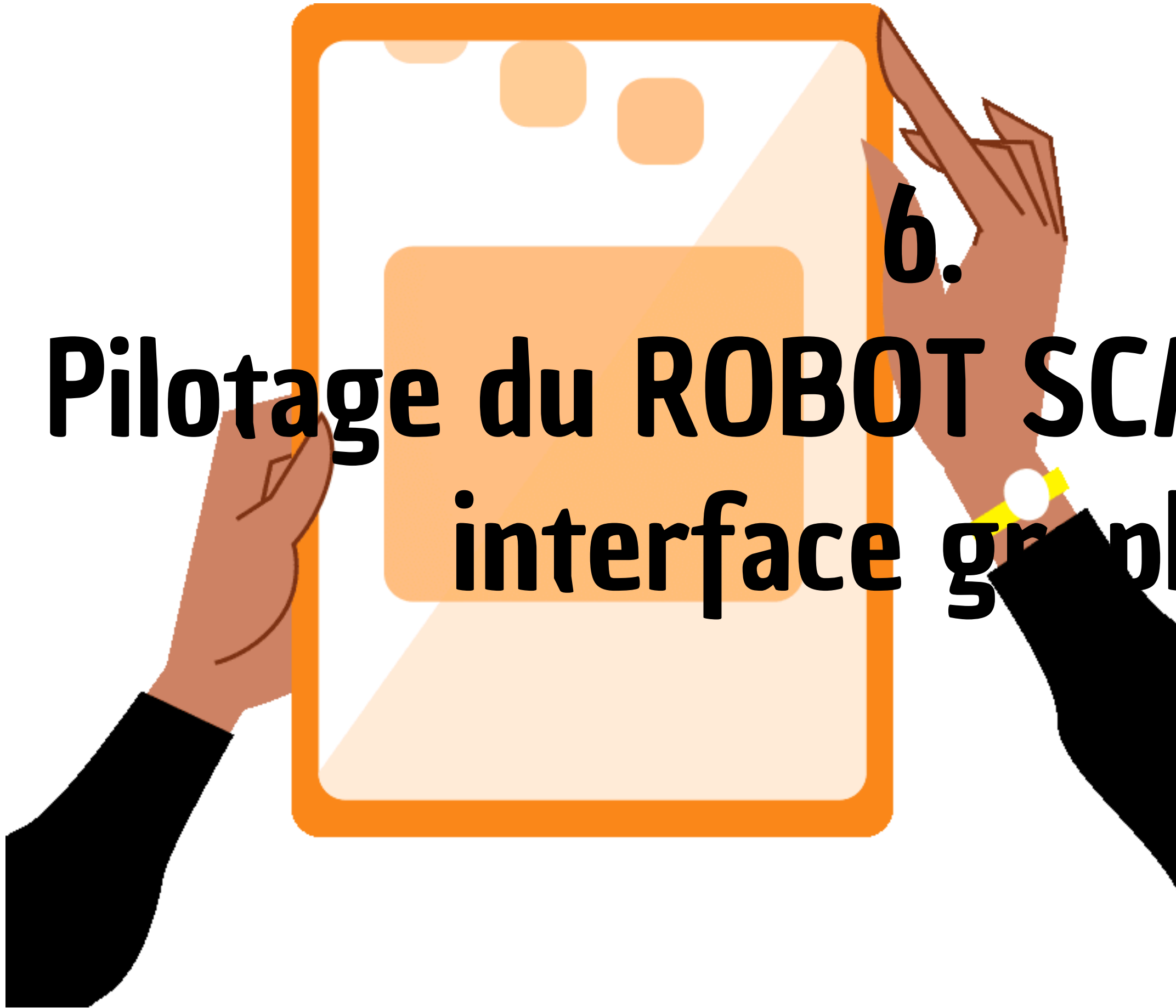
  void down() {
    Dynamixel.setEndless(3,ON,OFF);
    Dynamixel.turn(3,RIGHT,1000);
    delay(150);
    Dynamixel.turn(3,RIGHT,0);
    vr=0;
  }

}

```

6.

# Pilotage du ROBOT SCARA avec une interface graphique



## Coordonnées Cartésiennes

10 COORDONNEE X

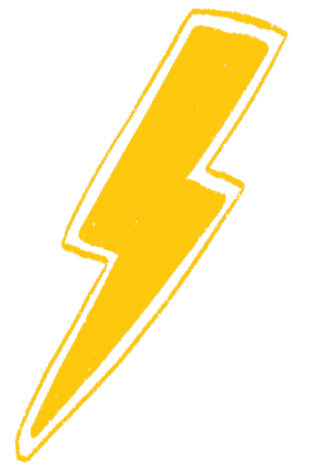
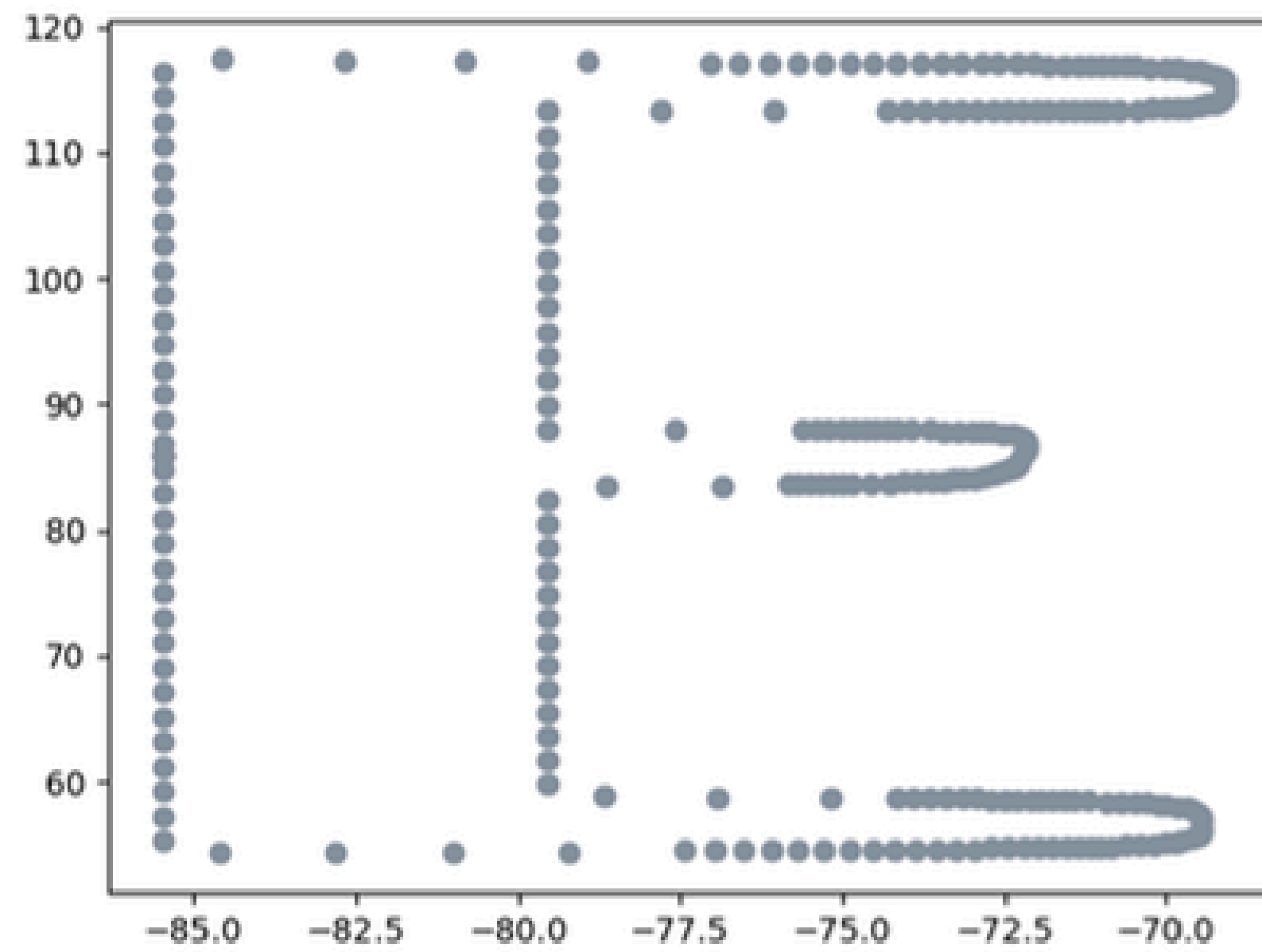
10 COORDONNEE Y

1 COORDONNEE Z

COMMANDE

## FICHER TEXTE

Plot



```
1  #%%
2  from tkinter import *
3  from tkinter import ttk
4  import tkinter as tk
5  import serial
6  import time
7  from matplotlib.backends.backend_tkagg import (FigureCanvasTkAgg , NavigationToolbar2Tk)
8  from matplotlib.figure import Figure
9  from matplotlib import pyplot as plt,animation
10 import numpy as np
11 import pandas as pd
12
13 ser=serial.Serial('com3',9600)
14 time.sleep(2)
15 ani=None
16 root=tk.Tk()
17 root.minsize(700,700)
18
19
20
21 # PERSONALISER
22 root.title("Interface Robot SCARA")
23 root.geometry("400x400")
24 #root.minsize(400,400)
25 root.config(background='#85929E')
26
27
```

```

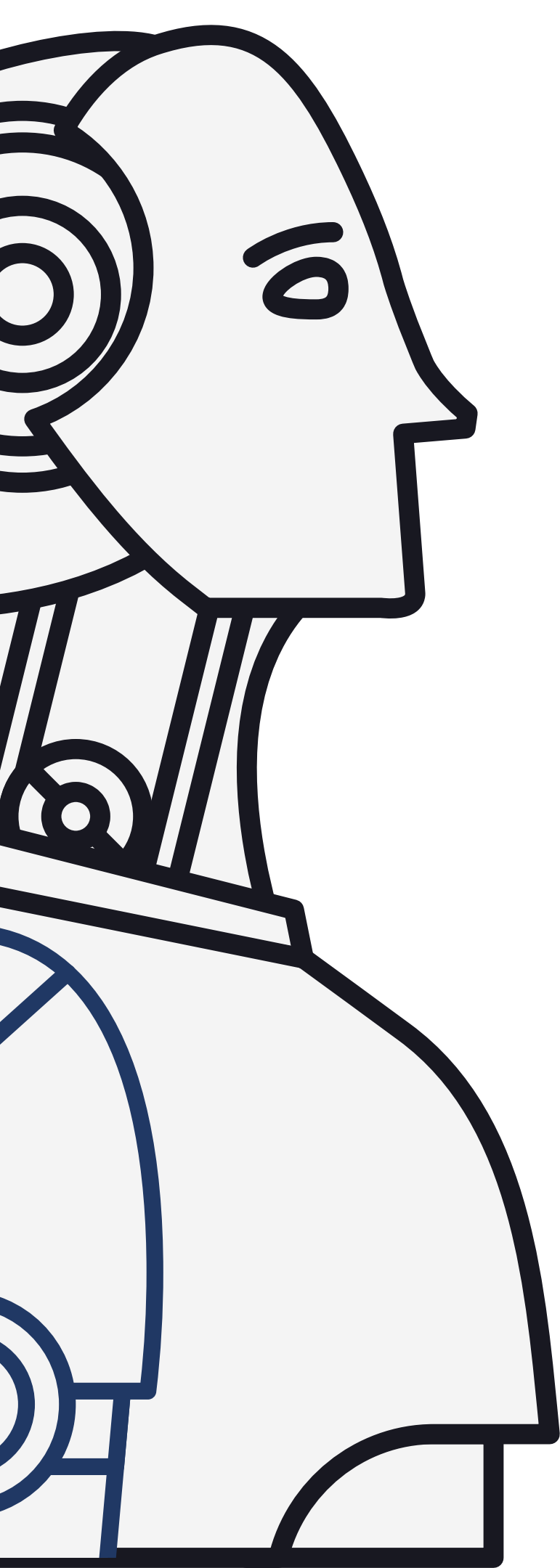
28 # FICHIER TEXTE
29 label3=StringVar()
30 label3.set("FICHIER TEXTE ")
31 labeld3=Label(root, textvariable=label3, height=4,bg='#85929E',font=("Courier", 16, "italic"))
32 labeld3.place(x=500,y=1)
33
34 fig = plt.Figure(dpi = 100)
35 plot1 = fig.add_subplot(111)
36 plot1.grid()
37 plot1.set_ylim(-125,125)
38 plot1.set_xlim(-125, 125)
39 canvas = FigureCanvasTkAgg(fig,
40 | | | | | | | | | | master = root)
41 toolbar=NavigationToolbar2Tk(canvas,root,pack_toolbar=False)
42 toolbar.update()
43 canvas.get_tk_widget().place(x=400,y=150)
44
45
46
47
48 X=[]
49 Y=[]
50 Z=[]
51 f=pd.read_csv("C:\\Users\\pc\\Desktop\\SCARA\\soutenance finale\\emines2.txt",delimiter="; ")
52 X=f.drop(["Y","Z"],axis=1)
53 Y=f.drop(["X","Z"],axis=1)
54 Z=f.drop(["X","Y"],axis=1)
55 X=np.array(X)
56 Y=np.array(Y)
57 Z=np.array(Z)
58
59 def anim(i):
60     plot1.scatter(X[i],Y[i],color='#85929E')
61     a=float(X[i])
62     b=float(Y[i])
63     c=int(Z[i])
64     ser.write((str(a)+' '+str(b)+' '+str(c)+'\n').encode())
65     time.sleep(2)
66

```

```

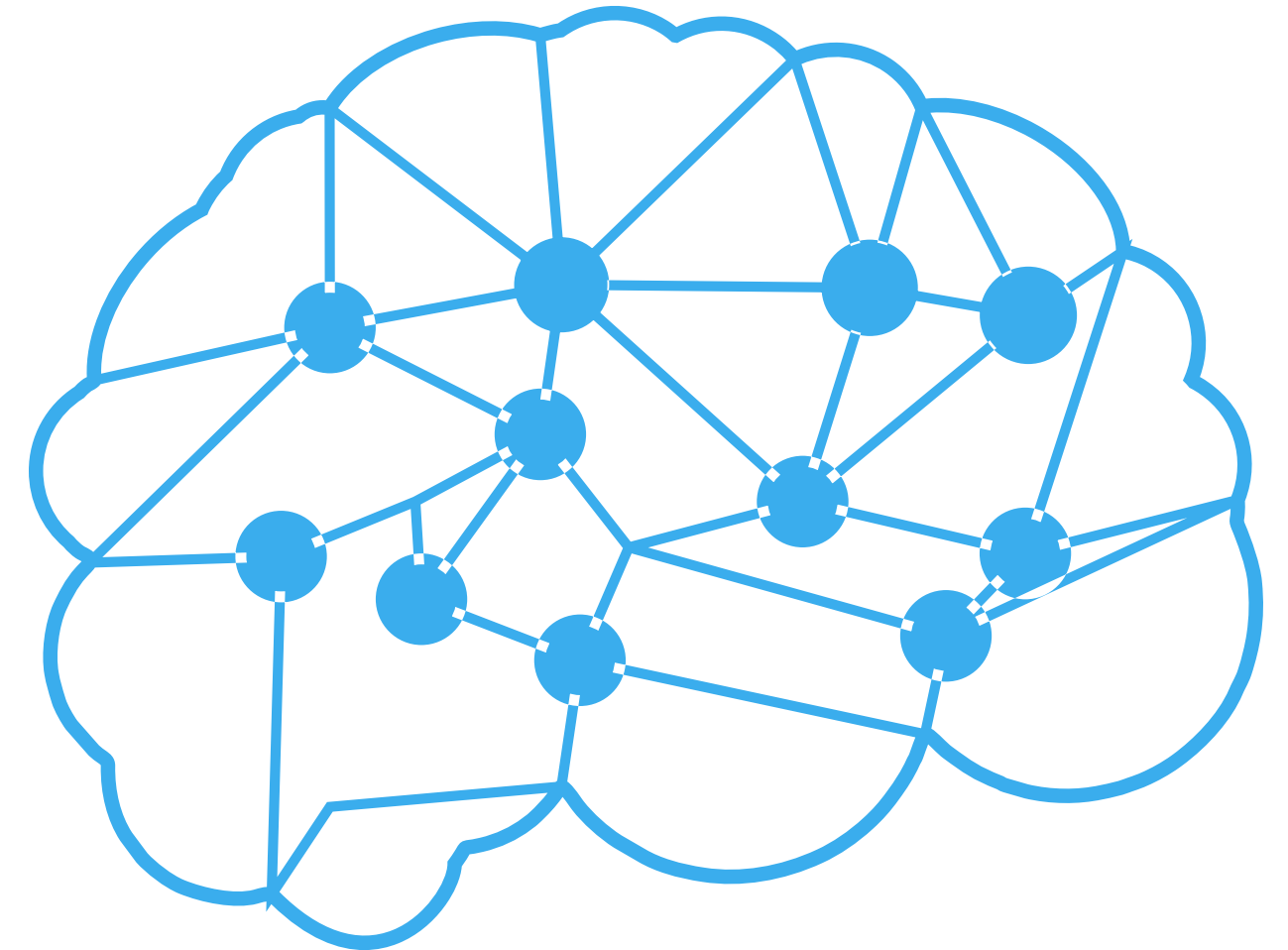
67 def g(plot1):
68     global ani
69     ani = animation.FuncAnimation(fig, anim, interval=2000,blit=False)
70     canvas.draw()
71
72 plot_button = Button(master = root,  command = lambda :g(plot1),text = "Plot")
73 plot_button.place(x=550,y=80)
74
75
76
77 # ENVOIE DE COMMANDE CARTESIENNE
78 def comm(plot1) :
79     plot1.clear()
80     ser.write((Xvar.get()+';'+yvar.get()+';'+zvar.get()+'\n').encode())
81     plot1.scatter(float(Xvar()),float(yvar()))
82     plot1.clear()
83     label=StringVar()
84     label.set("Coordonnées Cartésiennes ")
85     labeld=Label(root, textvariable=label, height=4,bg='#85929E',font=("Courier", 16, "italic"))
86     labeld.place(x=50,y=1)
87
88
89 Xvar=tk.StringVar()
90 XEntered=ttk.Entry(root,textvariable=Xvar)
91 XEntered.place(x=50,y=80)
92
93     labelText=StringVar()
94     labelText.set("COORDONNEE X")
95     labelDir=Label(root, textvariable=labelText, height=4,bg='#85929E')
96     labelDir.place(x=200,y=60)
97
98
99
100     yvar=tk.StringVar()
101     yEntered=ttk.Entry(root,textvariable=yvar)
102     yEntered.place(x=50,y=110)
103
104     labelText2=StringVar()
105     labelText2.set("COORDONNEE Y")
106     labelDir2=Label(root, textvariable=labelText2, height=2,bg='#85929E')
107     labelDir2.place(x=200,y=105)
108
109
110     zvar=tk.StringVar()
111     zEntered=ttk.Entry(root,textvariable=zvar)
112     zEntered.place(x=50,y=140)
113
114     labelText3=StringVar()
115     labelText3.set("COORDONNEE Z")
116     labelDir3=Label(root, textvariable=labelText3, height=2,bg='#85929E')
117     labelDir3.place(x=200,y=133)
118
119
120     buttoncommande=ttk.Button(root,text="COMMANDE",command=lambda:comm(plot1))
121     buttoncommande.place(x=150,y=200)
122

```

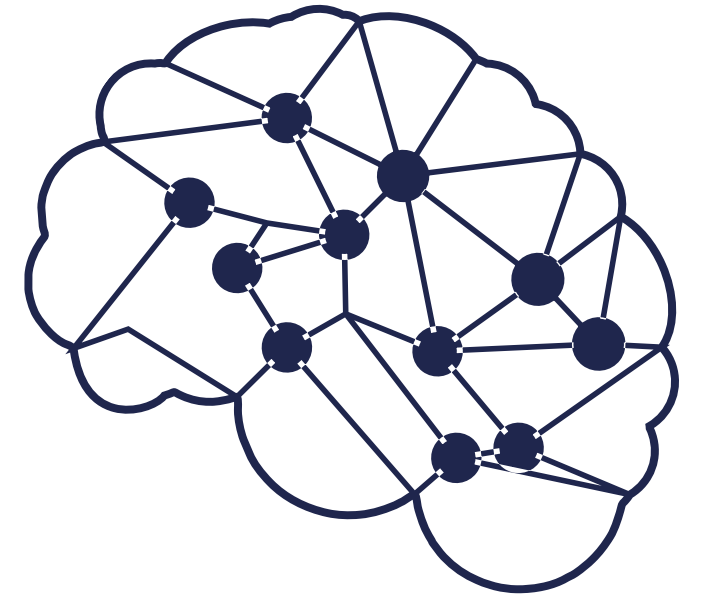


**7.**

**Commande par AI**



# Régression Linéaire



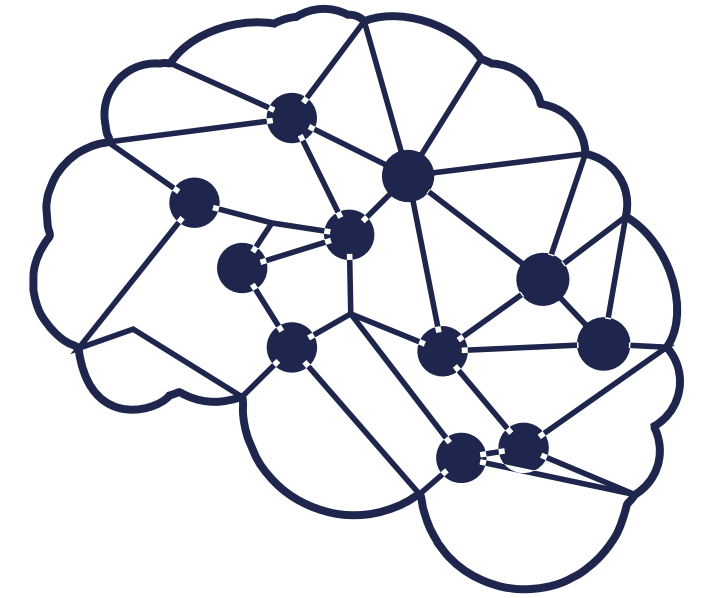
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_regression
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
from mpl_toolkits import mplot3d
import pandas as pd
```

```
DATA=[]
Y=[]
f=pd.read_csv("data_dyna_180 (1).txt",delimiter=";")
Y=f.drop(['alpha1 dyna','alpha2 dyna'],axis=1)
DATA=f.drop(['X(en mm)','Y(en mm)'],axis=1)
```

```
Y_train,Y_test,DATA_train,DATA_test= train_test_split(Y,DATA,test_size=0.2)
model=LinearRegression()
model.fit(Y_train,DATA_train)
model.score(Y_test,DATA_test)
```

0.5365902428498864

# K-Nearest Neighbors



```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsRegressor
from sklearn.model_selection import train_test_split
import pandas as pd
```

```
Data=[] # Initialisation de la matrice des angles
Y=[] # Initialisation de la matrice des coordonnées x ,y
f=pd.read_csv("data_dyna_Knn.csv",delimiter=";")
Y=f.drop(['alpha1 dyna','alpha2 dyna'],axis=1)
DATA=f.drop(['X(en mm)','Y(en mm)'],axis=1)
```

```
model=KNeighborsRegressor(1)
model.fit(Y,DATA)
model.score(Y,DATA)
```

```
Y_train,Y_test,DATA_train,DATA_test= train_test_split(Y,DATA,test_size=0.25)
```

```
position_finale=np.array((220,0))
prediction_knn=model.predict(position_finale.reshape(1, -1))[0]
print("Angle dynamixel :",prediction_knn)
print("Angle 01 :",270-prediction_knn*360/4095)
print("Angle 02 :",180-prediction_knn*360/4095)
```

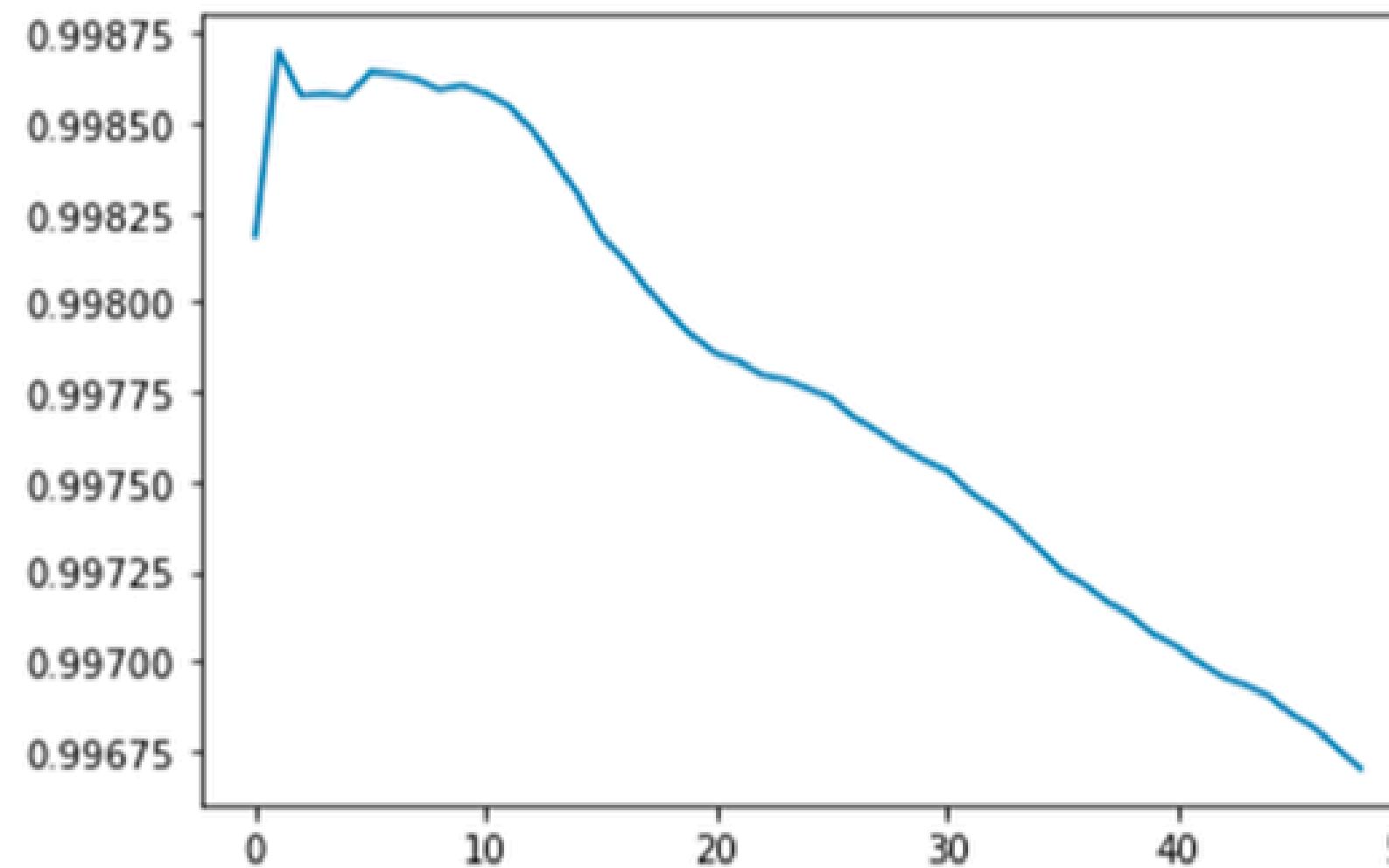
```
Angle dynamixel : [3078.1  2040.65]
Angle 01 : -0.602197802197793
Angle 02 : 0.602197802197793
```

# Validation Set

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsRegressor
from sklearn.model_selection import train_test_split
import pandas as pd

Data=[] # Initialisation de la matrice des angles
Y=[] # Initialisation de la matrice des coordonnées x ,y
f=pd.read_csv("data_dyna_180 (1).txt",delimiter=";")
Y=f.drop(['alpha1 dyna','alpha2 dyna'],axis=1)
DATA=f.drop(['X(en mm)','Y(en mm)'],axis=1)

Y_train,Y_test,DATA_train,DATA_test= train_test_split(Y,DATA,test_size=0.25)
from sklearn.model_selection import cross_val_score
#cross_val_score(KNeighborsRegressor(1),Y_train,DATA_train,cv=5)
val_score=[]
for k in range (1,50):
    score=cross_val_score(KNeighborsRegressor(k),Y_train,DATA_train,cv=5).mean()
    val_score.append(score)
plt.plot(val_score)
print(max(val_score))
```



# Réseau de neurones

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neural_network import MLPRegressor
from sklearn.model_selection import train_test_split
```

```
DATA=[]
Y=[]
f=pd.read_csv("data_dyna_180 (1).txt",delimiter=";")
Y=f.drop(['alpha1 dyna', 'alpha2 dyna'],axis=1)
DATA=f.drop(['X(en mm)', 'Y(en mm)'],axis=1)
```

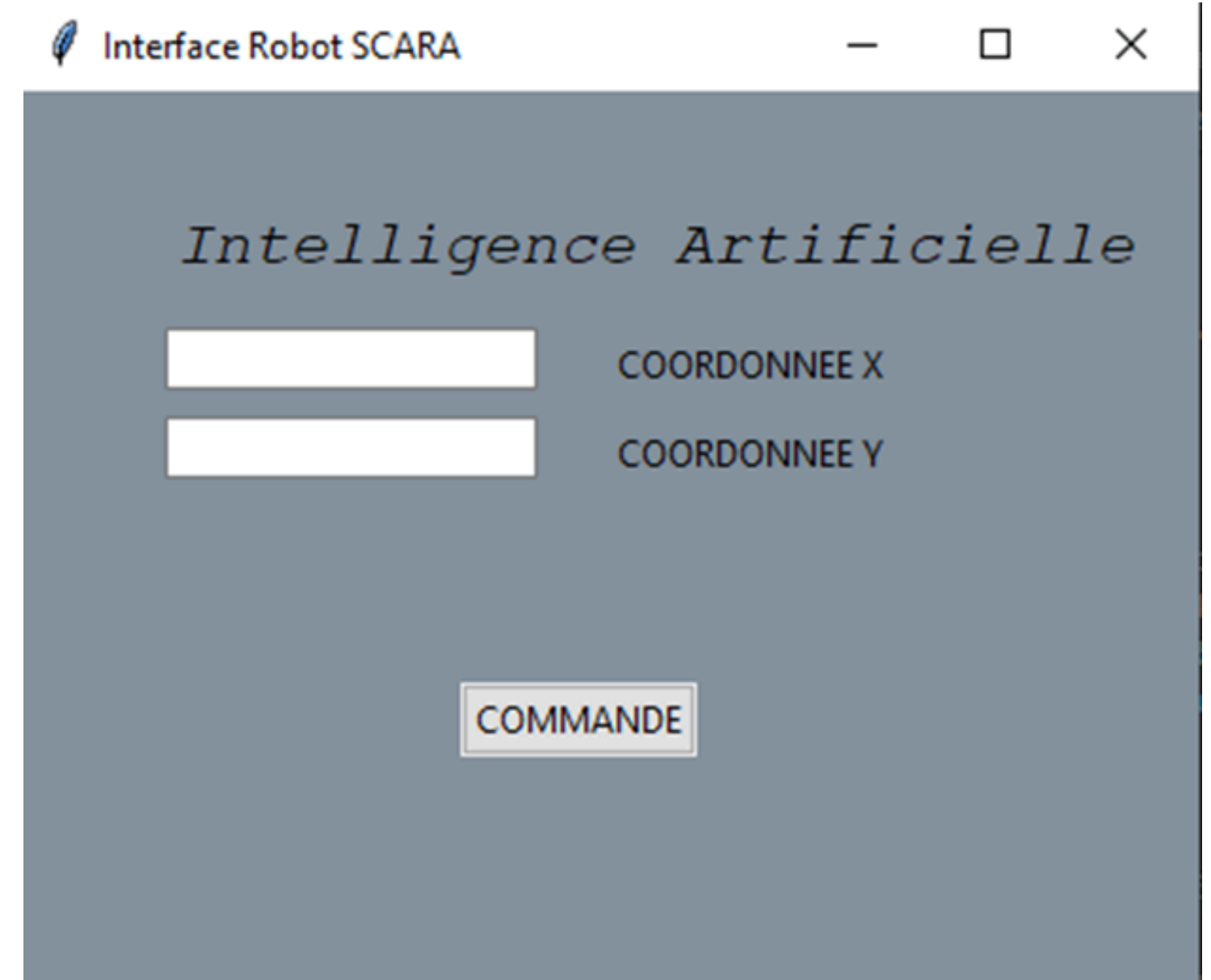
```
mlp_reg = MLPRegressor(hidden_layer_sizes=(50,50),
                        max_iter = 500,activation = 'relu',
                        solver = 'adam')
```

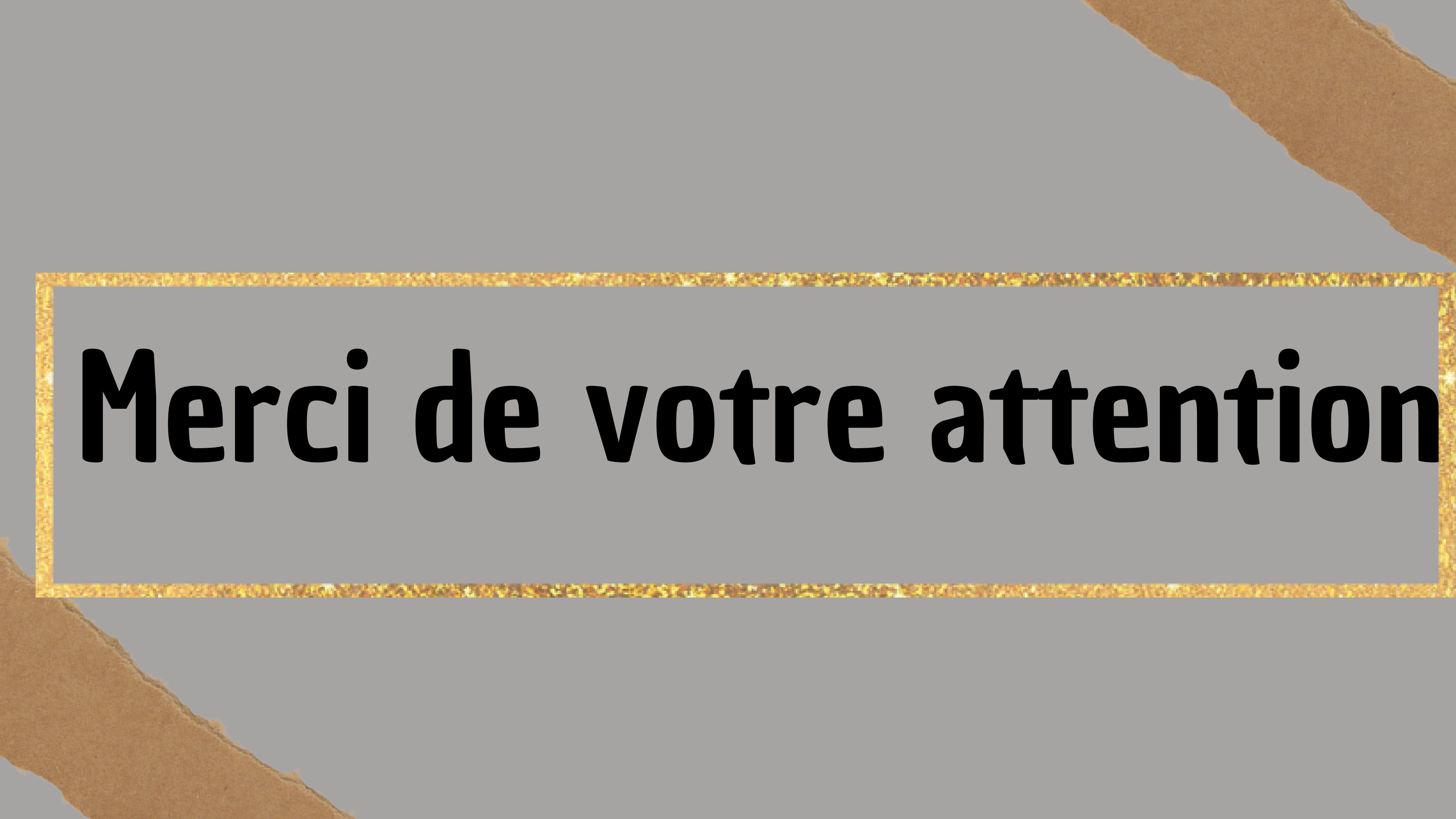
```
mlp_reg.fit(Y, DATA)
```

```
MLPRegressor(hidden_layer_sizes=(50, 50), max_iter=500)
```

```
mlp_reg.score(Y,DATA)
```

```
0.9960452462039935
```





**Merci de votre attention**